

simple feature specification of the Open GeoSpatial Consortium. This enables you to develop GIS based applications (<https://www.monetdb.org/Documentation/Extensions/GIS>).

- **LifeScience:** MonetDB has the SAM/BAM module, which is among the prominent standards for managing the DNA sequence alignment data (<https://www.monetdb.org/bam>).
- **Data vaults:** The latest databases are not restricted only to tabular data. They are often required to handle various files as well. MonetDB data vaults are database-linked external files (<https://www.monetdb.org/Documentation/Extensions/DataVaults>).

Language bindings

MonetDB is loaded with Python, Perl, Ruby, PHP, JDBC and ODBC interface libraries. The important point to be noted is that these interfaces are native implementations and hence do not require the installation of MonetDB client/server code. Apart from this, there is a *MonetDB.R* connector (<http://monetr.r-forge.r-project.org/>), which makes the interaction with MonetDB from R simple and elegant.

This article only explores support. MonetDB has a native Python client API. The support is provided for Python 2.7 and 3.3. This API is compatible with Python DBAPI 2.0.

The PymonetDB can be installed with *pip* using the following command:

```
pip install pymonetdb
```

The interaction from Python is very simple, as shown below:

```
> # import the SQL module
> import pymonetdb
>
> # set up a connection. arguments below are the defaults
> connection = pymonetdb.connect(username="monetdb",
password="monetdb", hostname="localhost", database="demo")
>
> # create a cursor
> cursor = connection.cursor()
>
> # increase the rows fetched to increase performance
(optional)
> cursor.arraysize = 100
>
> # execute a query (return the number of rows to fetch)
> cursor.execute('SELECT * FROM tables')
26
>
> # fetch only one row
> cursor.fetchone()
```

```
(1062, 'schemas', 1061, None, 0, True, 0, 0)
>
> # fetch the remaining rows
> cursor.fetchall()
[(1067, 'types', 1061, None, 0, True, 0, 0),
 (1076, 'functions', 1061, None, 0, True, 0, 0),
 (1085, 'args', 1061, None, 0, True, 0, 0),
 (1093, 'sequences', 1061, None, 0, True, 0, 0),
 (1103, 'dependencies', 1061, None, 0, True, 0, 0),
 (1107, 'connections', 1061, None, 0, True, 0, 0),
 (1116, '_tables', 1061, None, 0, True, 0, 0),
 ...
 (4141, 'user_role', 1061, None, 0, True, 0, 0),
 (4144, 'auths', 1061, None, 0, True, 0, 0),
 (4148, 'privileges', 1061, None, 0, True, 0, 0)]
>
> # Show the table meta data
> cursor.description
[('id', 'int', 4, 4, None, None, None),
 ('name', 'varchar', 12, 12, None, None, None),
 ('schema_id', 'int', 4, 4, None, None, None),
 ('query', 'varchar', 168, 168, None, None, None),
 ('type', 'smallint', 1, 1, None, None, None),
 ('system', 'boolean', 5, 5, None, None, None),
 ('commit_action', 'smallint', 1, 1, None, None, None),
 ('temporary', 'tinyint', 1, 1, None, None, None)]
```

The MAPI library provides the lowest level interface. Sample code to interact through MAPI is shown below:

```
> from pymonetdb import mapi
> mapi_connection = mapi.Connection()
> mapi_connection.connect(hostname="localhost", port=50000,
username="monetdb", password="monetdb", database="demo",
language="sql", unix_socket=None, connect_timeout=-1)
> mapi_connection.cmd("sSELECT * FROM tables;")
```

To summarise, MonetDB is a time tested choice and is worth exploring if you are interested in column based databases. **END** 🐧

References

- [1] <https://www.monetdb.org/>
- [2] <http://monetr.r-forge.r-project.org/>
- [3] <https://www.monetdb.org/Documentation>

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry.